

University of Pittsburgh

Introduction to Operating Systems
CS 1550

Spring 2022

Sherif Khattab

ksm73@pitt.edu

(Some slides are from **Silberschatz, Galvin and Gagne ©2013)**

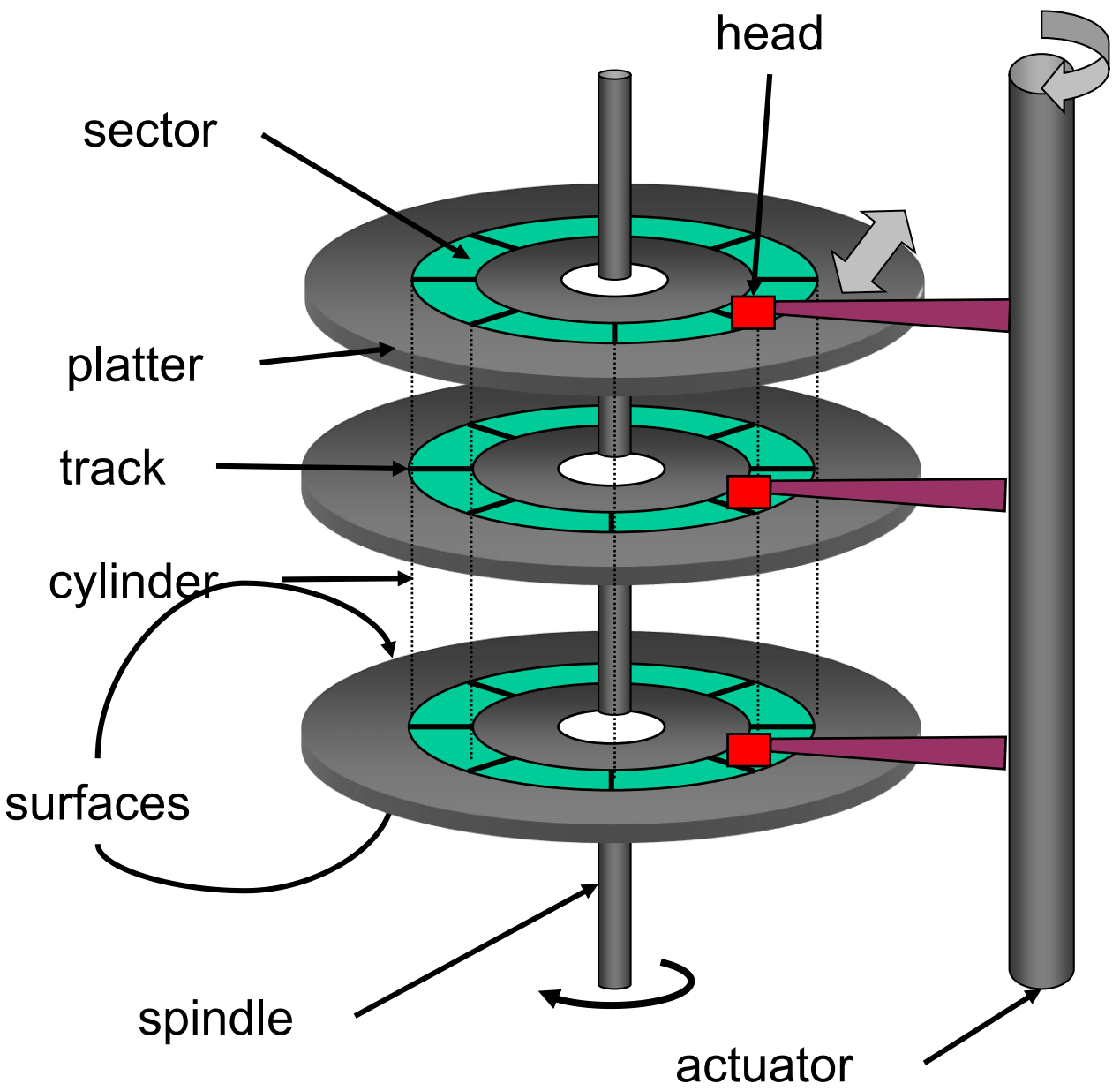
Announcements

- Upcoming deadlines:
  - Homework 11: due on 4/18
  - Project 3: Late due date on 4/13
    - Late penalty waived
  - Lab 4: due on 4/15
  - Quiz 3: due on 4/15
  - Homework 12: due on 4/25
  - Lab 5: due on 5/2
  - Project 4 and Quiz 4: due on 5/2
  - Bonus Homework: due on 5/2
  - Course Post-Test: due on 5/2

Muddiest Points (1/2)

- when is the parity checked to see if there is a bit error
- a little confused about parity example
- How to flip the correct bits back that were changed with parity?
- What is parity used for
- ECC
- What things move how using what in the disk
- disk request time delay factors
  - transfer time/rotating time/seek time
- I feel like rotational delay and actual transfer time are overlapped. The sector is spending time to rotate to the head, and at the same time, the head is rotating to find that sector. Are they moving towards each other at the same time?

Disk drive structure



Muddiest Points (2/2)

- In the last top hat question, why the page table gets smaller if the page frames are larger. I understand that would mean less entries, but wouldn't each entry would be larger, making the overall table the same size?
- What are RAIDs for?
- Does RAID4 have any advantages over RAID5?
- All of it, bro

Problem of the Day – Part 1

- How does a file system handle errors?
- Answer: Defense in Depth
  - multiple layers of error detection/correction

When good disks go bad…

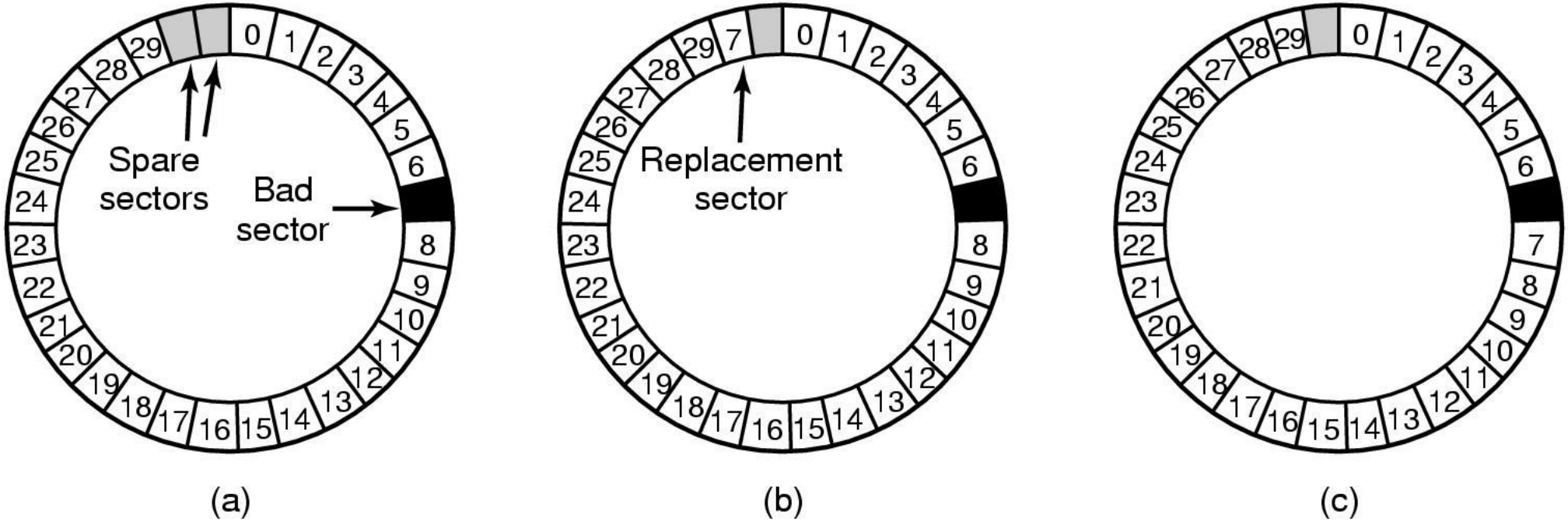
Disks have defects

In 3M+ sectors, this isn't surprising!

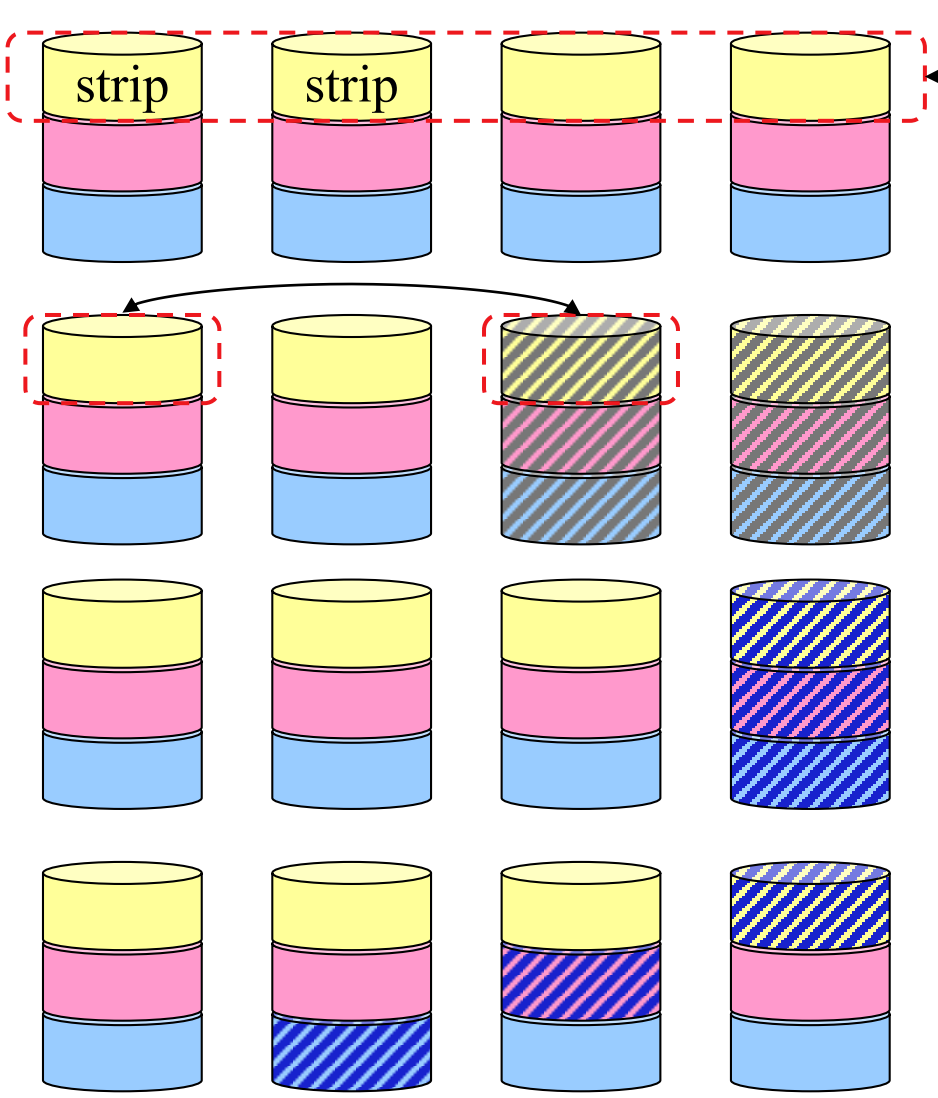
ECC helps with errors, but sometimes this isn't enough

Disks keep spare sectors (normally unused) and remap bad sectors into these spares

If there's time, the whole track could be reordered…



RAIDs, RAIDs, and more RAIDs



RAID 0
(Redundant Array of Inexpensive Disks

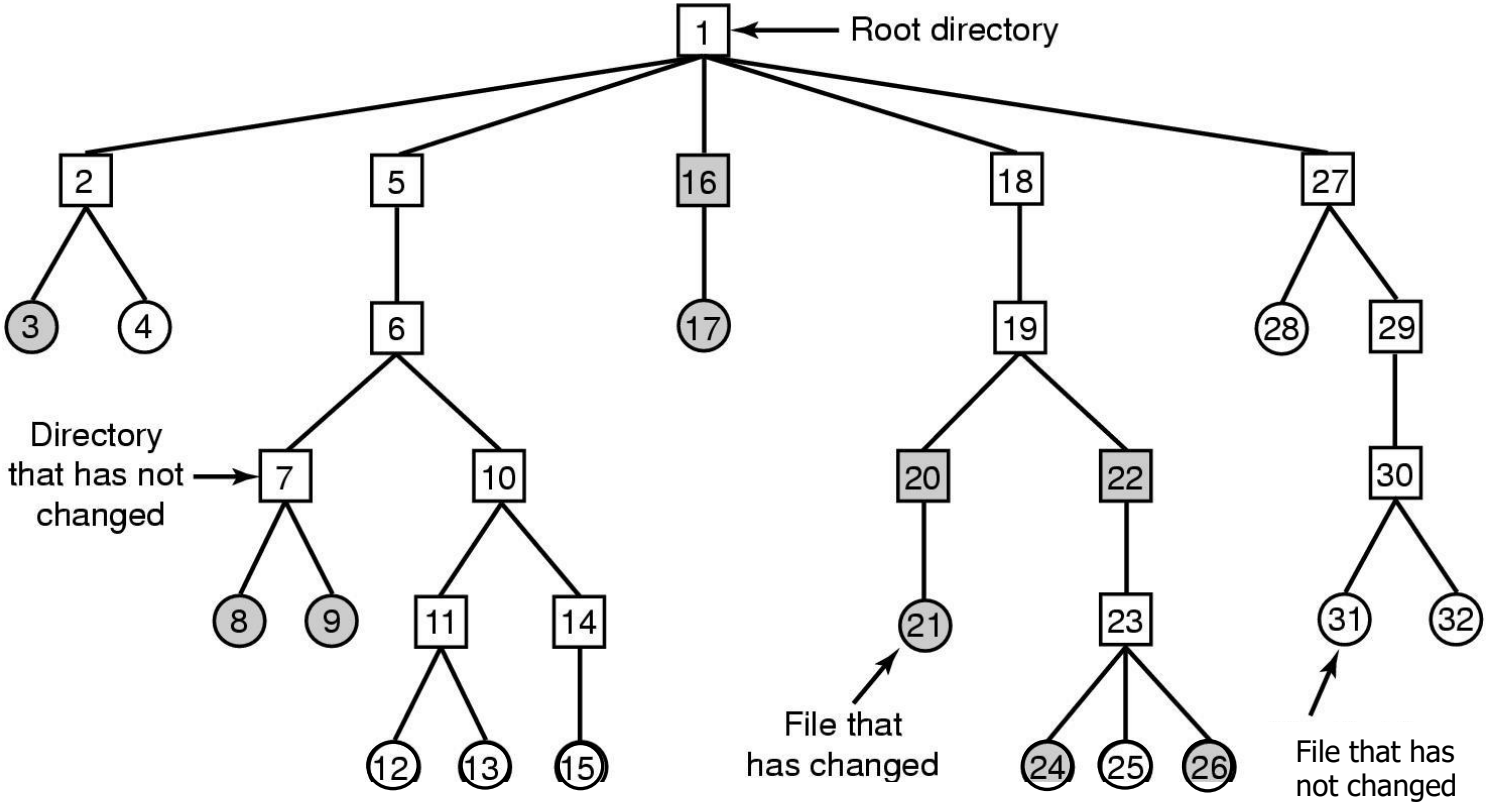
RAID 1
(Mirrored copies)

RAID 4
(Striped with parity)

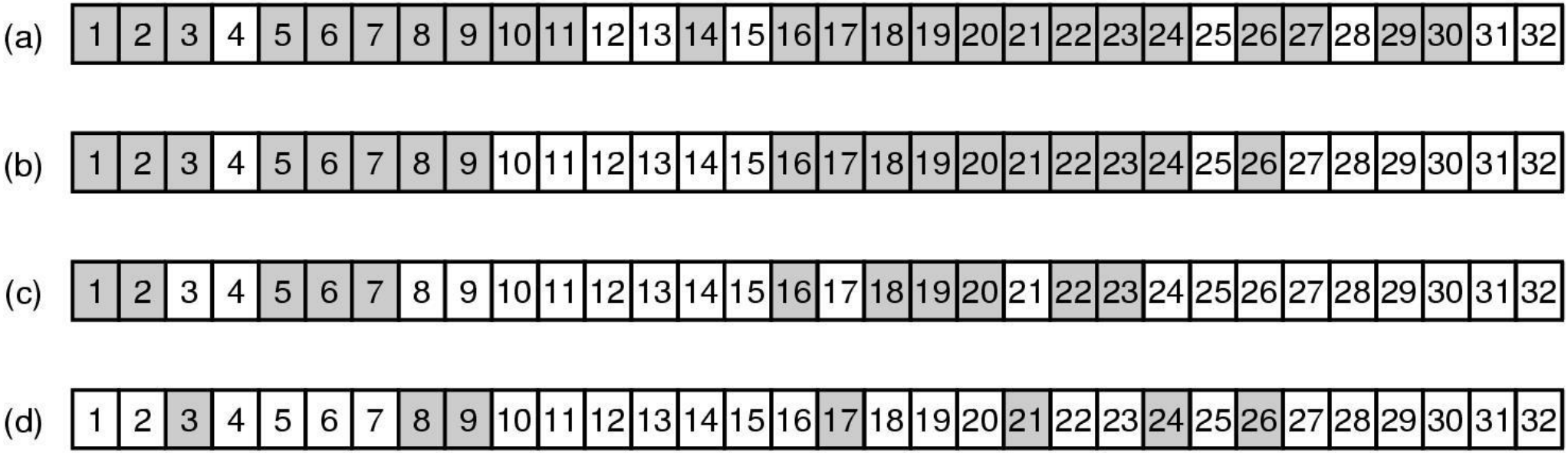
RAID 5
(Parity rotates through disks)

Backing up a file system

- A file system to be dumped
  - Squares are directories, circles are files
  - Shaded items, modified since last dump
  - Each directory & file labeled by i-node number

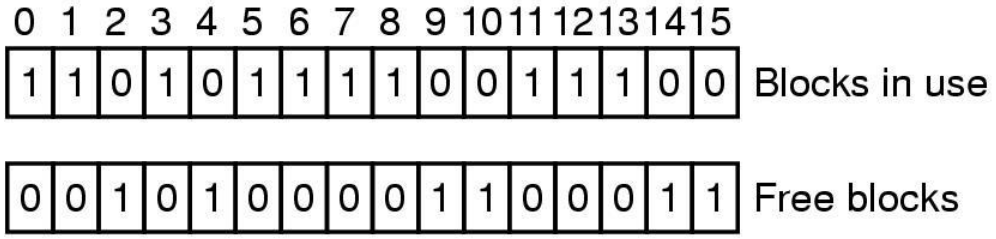


Bitmaps used in a file system dump



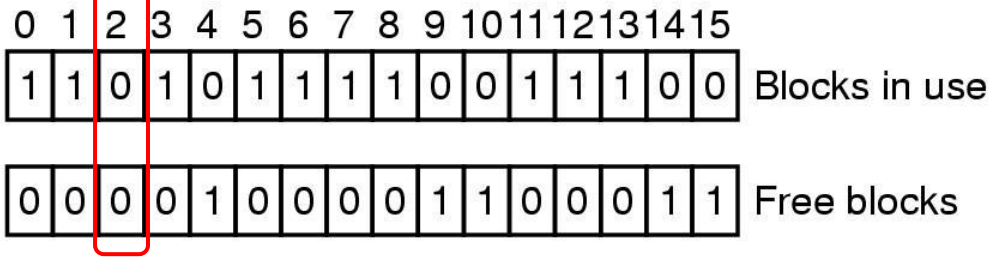
Checking the file system for consistency

Consistent

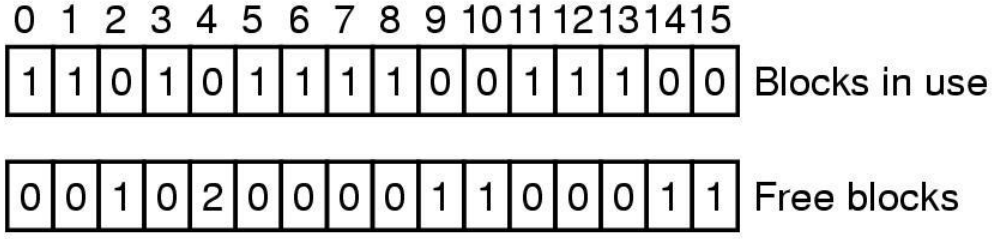


(a)

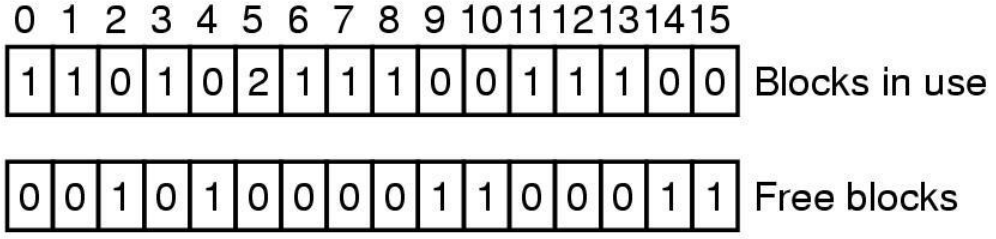
Missing ("lost") block



(b)



Duplicate block in free list

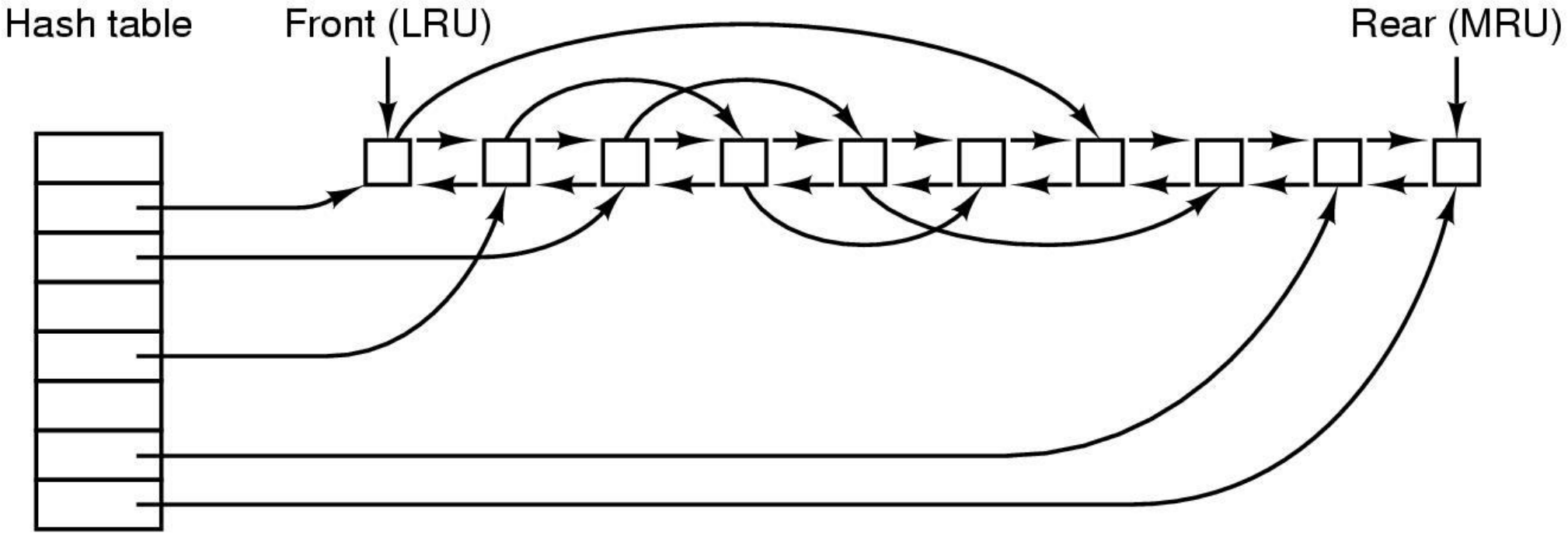


Duplicate block in two files

Problem of the Day – Part 2

- How does a file system hide disk access delays?
- Answer: Caching

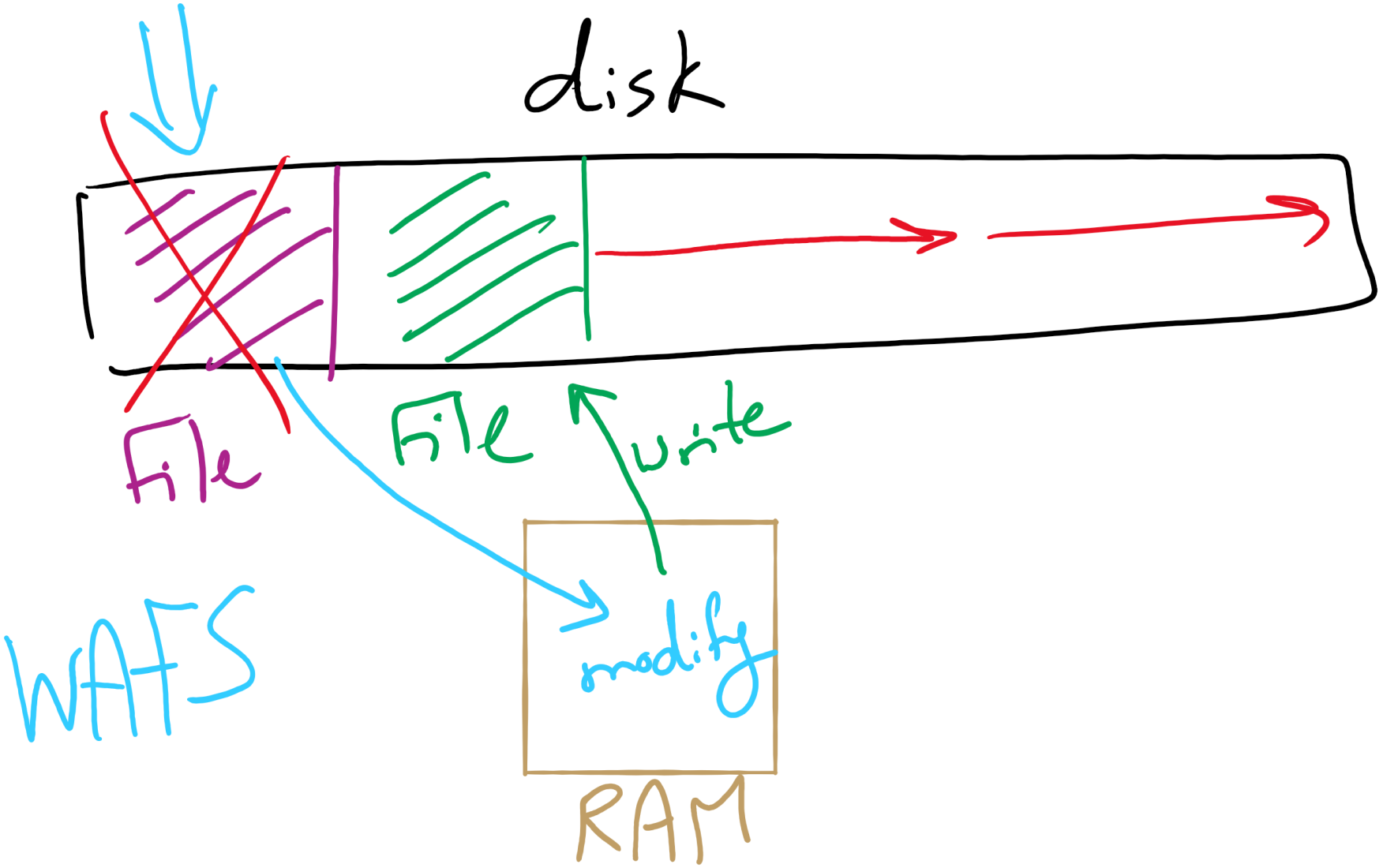
File block cache data structures



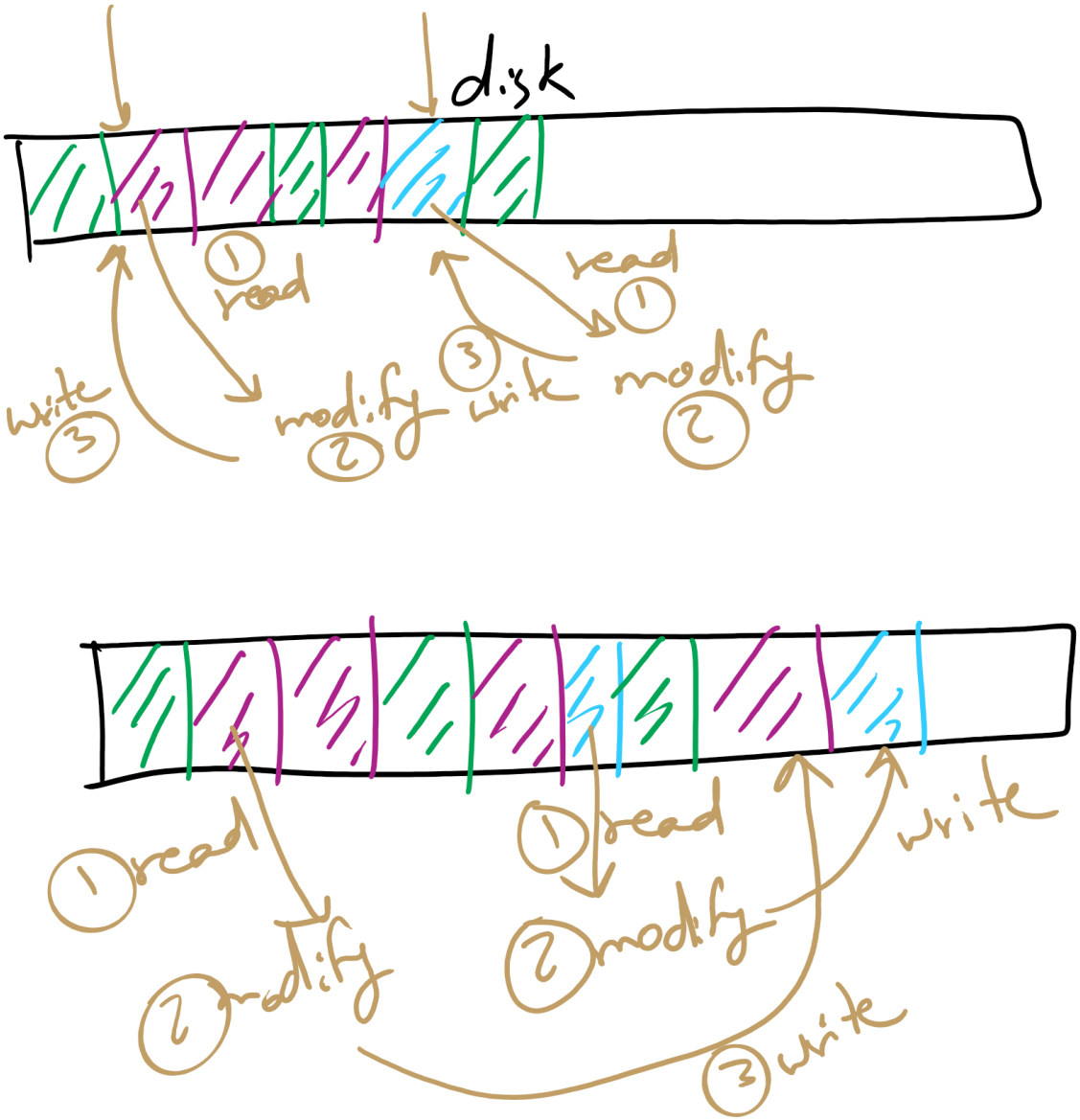
Log-structured file systems

- Trends in disk & memory
  - Faster CPUs
  - Larger memories
- Result
  - More memory -> disk caches can also be larger
  - Increasing number of read requests can come from cache
  - Thus, most disk accesses will be writes
- LFS structures entire disk as a log
  - All writes initially buffered in memory
  - Periodically write these to the end of the disk log
  - When file opened, locate i-node, then find blocks
- Issue: what happens when blocks are deleted?

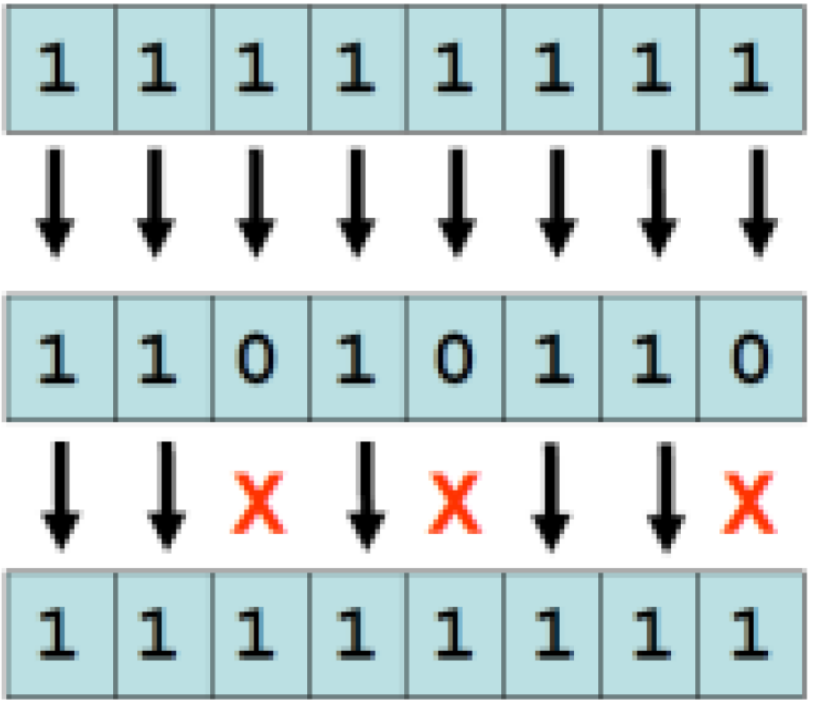
Log Structured File System



Log Structured File System



Flash File System



Start off with all 1's

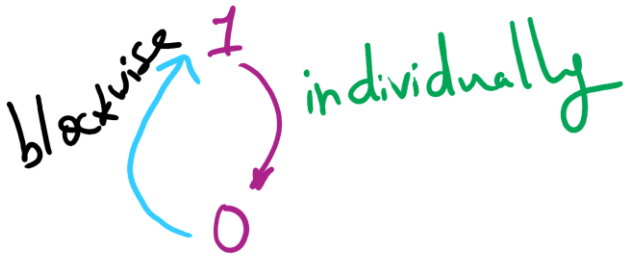
Programming from 1's to 0's is allowed

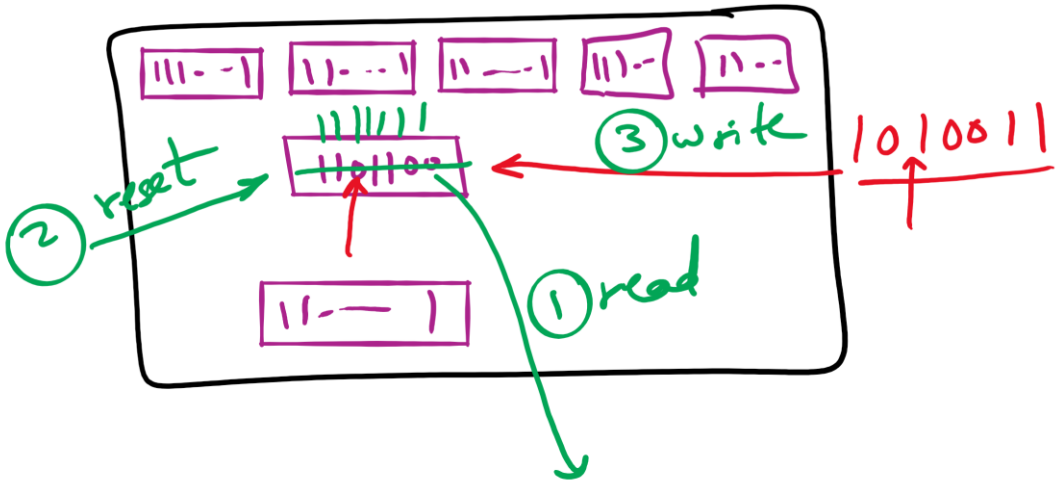
Programming from 0's to 1's is **not** allowed

Wear Leveling

Count total writes per flash sector and attempt to balance across the whole disk

Wear leveling for SSDs





1 → 0 : individually

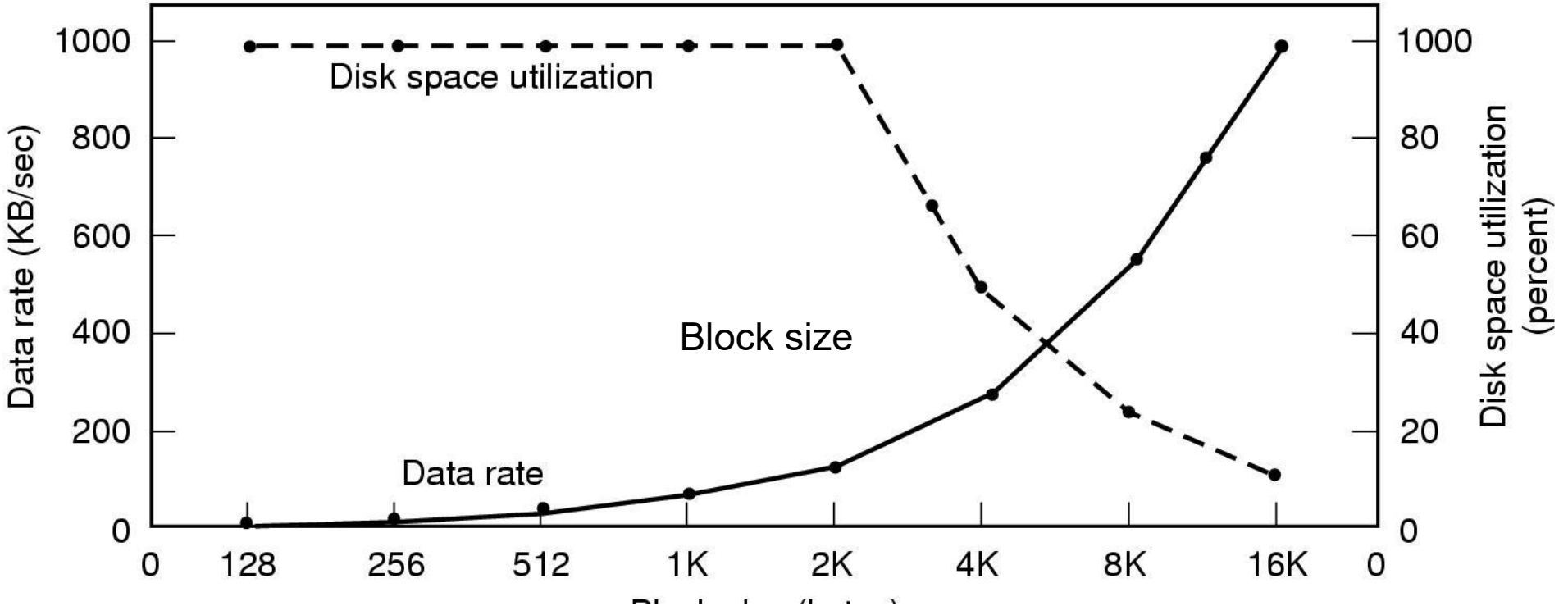
0 → 1 : read block → reset block
→ write block

wear leveling

read block → write to a new block

Disk block size

- Dark line (left hand scale) gives data rate of a disk
- Dotted line (right hand scale) gives disk space efficiency
- All files 2KB

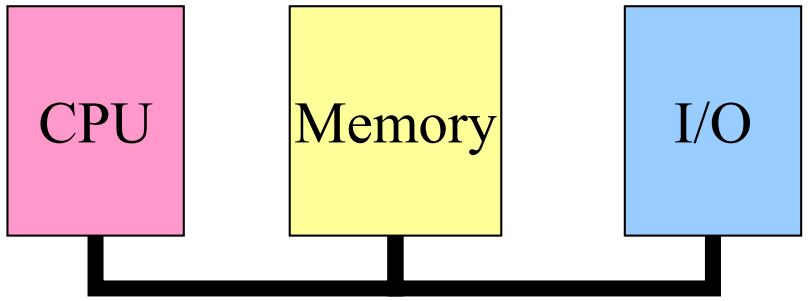


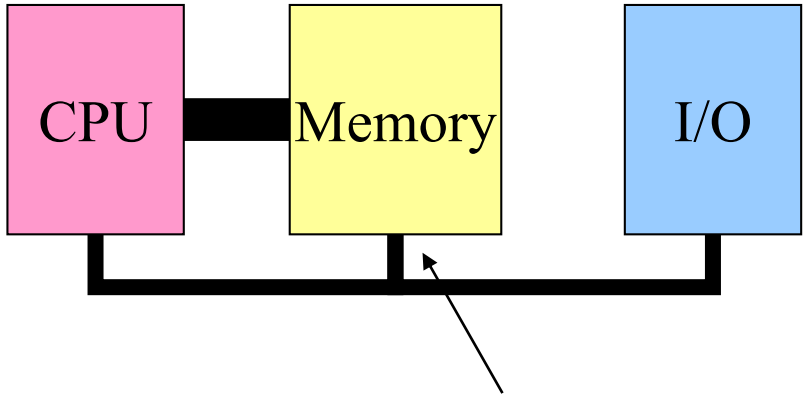
Problem of the Day – Part 3

- How do device drivers program I/O devices?
- Answer: three methods
  - polling
  - Interrupts
  - DMA

How are I/O devices connected?

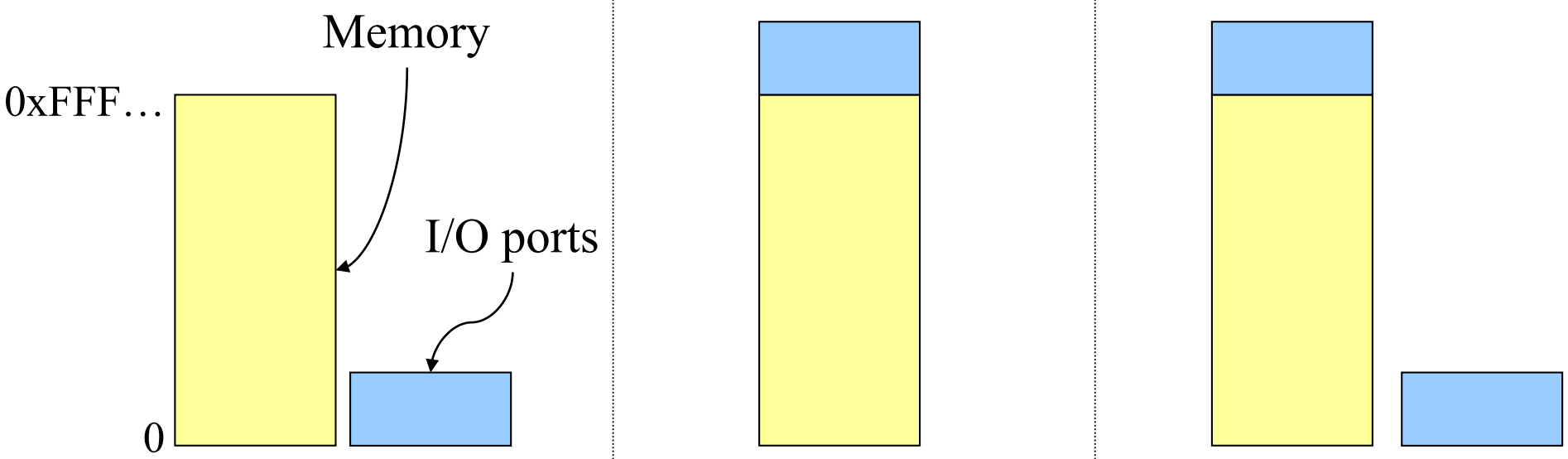
- Single-bus
  - All memory accesses go over a shared bus
  - I/O and RAM accesses compete for bandwidth
- Dual-bus
  - RAM access over high-speed bus
  - I/O access over lower-speed bus
  - Less competition
  - More hardware (more expensive…)





This port allows I/O devices access into memory

Memory-Mapped vs. separate I/O Space



Separate I/O & memory space

Memory-mapped I/O

Hybrid: both memory-mapped & separate spaces

Example: Dynamic Frequency on XScale

```
// Allocate some space for the virtual reference to CCCR
LPVOID virtCCCR = VirtualAlloc(0, sizeof(DWORD), MEM_RESERVE, PAGE_NOACCESS);

//0x41300000 is the memory-mapped location of the CCCR register
LPVOID CCCR = (LPVOID)(0x41300000 / 256); // shift by 8 bits for ability to address 2^40 bytes

// Map writing the virtual pointer to the physical address of the CCCR register
VirtualCopy((LPVOID)virtCCCR, CCCR, sizeof(DWORD), PAGE_READWRITE | PAGE_NOCACHE | PAGE_PHYSICAL);

// Set the CCCR register with the new speed
*(int *)virtCCCR = new_speed;

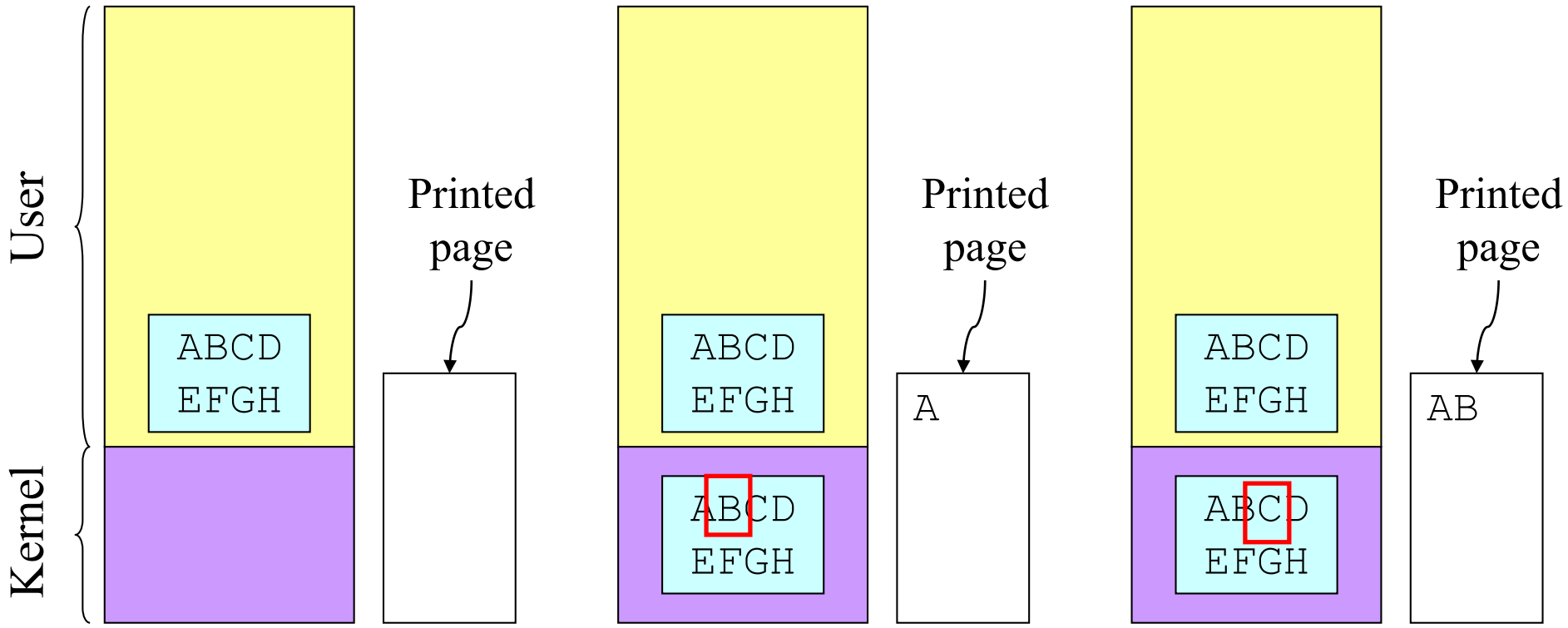
// Call the assembly function to actually perform the switch
doSwitch(0x02 | 0x01); //0x02 means turbo mode, 0x01 means the clock is being switched

// Clean up memory by freeing the virtual register.
VirtualFree(virtCCCR, 0, MEM_RELEASE);
virtCCCR = NULL;
```

```
; Coprocessor 14, register C6 (CLKCFG) initiates the changes programmed in CCCR
; when CLKCFG is written.

doSwitch
      MOV r3, r0                ; Move r0, the argument to doSwitch, into register r3
      MCR p14, 0, r3, c6, c0, 0 ; Copy the contents of r3 into register c6 on coprocessor 14.
      MOV pc, lr                ; return execution to where it last left off
```

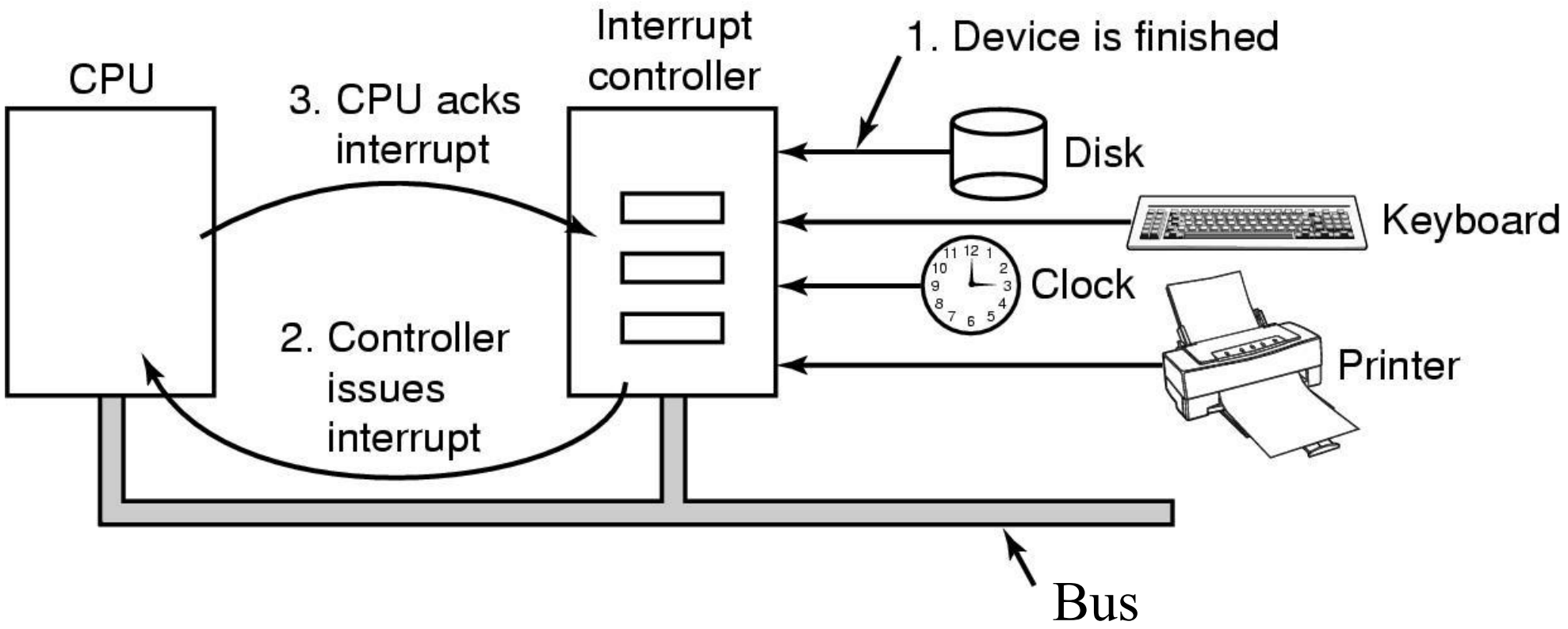
Programmed I/O example: printing a page



Polling

```
copy_from_user (buffer, p, count);  // copy into kernel buffer
for (j = 0; j < count; j++) {       // loop for each char
  while (*printer_status_reg != READY)
    ;                               // wait for printer to be ready
  *printer_data_reg = p[j];    // output a single character
}
return_to_user();
```

Hardware's view of interrupts



Interrupt-driven I/O

```
copy_from_user (buffer, p, count);
j = 0;
enable_interrupts();
while (*printer_status_reg != READY)
  ;
*printer_data_reg = p[0];
scheduler(); // and block user
```

Code run by system call

```
if (count == 0) {
  unblock_user();
} else {
  j++;
  *printer_data_reg = p[j];
  count--;
}
acknowledge_interrupt();
return_from_interrupt();
```

Code run at interrupt time
(Interrupt handler)